

Bug Fix Explanations

★ Bug 1: API Artificial Delay

- Location: backend/src/main/java/com/internal/tasktracker/TaskController.java (Line 37, Backend Layer)
- Discovery: Noticed slow API response times during testing, especially for shorter queries, which led to investigating the controller layer.
- Root Cause: An artificial `Thread.sleep()` was intentionally added to simulate query complexity based on the length of the query.
- Fix: Removed the `Thread.sleep()` block completely. This approach was chosen because production code should not have artificial delays; removing it significantly improves API performance and avoids potential thread pool exhaustion.

★ Bug 2: Incorrect SQL Logic (Operator Precedence)

- Location: backend/src/main/java/com/internal/tasktracker/TaskRepository.java (Line 14, Database Layer) and db/oracle/task_search_package.sql (Lines 53, 67, Database Layer)
- Discovery: Observed that searching for a specific term could return archived tasks, effectively ignoring the `archived = FALSE` or status filters.
- Root Cause: Missing parentheses around the OR condition for title and description. Due to standard SQL operator

precedence, AND binds tighter than OR, causing the query to evaluate incorrectly. If the description matched, it ignored the archived or status checks.

- Fix: Added parentheses around (LOWER(title) LIKE :term OR LOWER(description) LIKE :term). This ensures the text search acts as a single group, and the archived and status conditions apply correctly to all results.

★ Bug 3: Pagination Reset on Filter Change

- Location: frontend/src/App.jsx (Lines 24-25, Frontend UI Layer)
- Discovery: Found that when on a higher page number (e.g., page 3) and typing a new search query that yields fewer total results, the table would show empty results because it was still requesting page 3.
- Root Cause: Changing the search query or status filter updated those specific states but did not reset the current page back to 1.
- Fix: Updated the onChange handlers for SearchBar and StatusFilter to call setPage(1) alongside updating their respective filter states. This guarantees the user is always returned to the first page of results whenever filters change.

★ Bug 4: Network Race Conditions and Missing Debounce

- Location: frontend/src/hooks/useTasks.js (Lines 11-30, Frontend Logic Layer)
- Discovery: Typing quickly in the search bar caused

multiple API requests to fire simultaneously. Sometimes older requests completed after newer ones, causing stale data to override the latest UI state.

- Root Cause: The `useEffect` hook for fetching tasks lacked debouncing and had no mechanism to ignore stale responses.
- Fix: Implemented a 300ms debounce using `setTimeout` and introduced an `ignore` boolean flag in the cleanup function. This approach reduces unnecessary network load by waiting for typing to pause and guarantees that outdated fetch callbacks are ignored, effectively resolving the race condition.